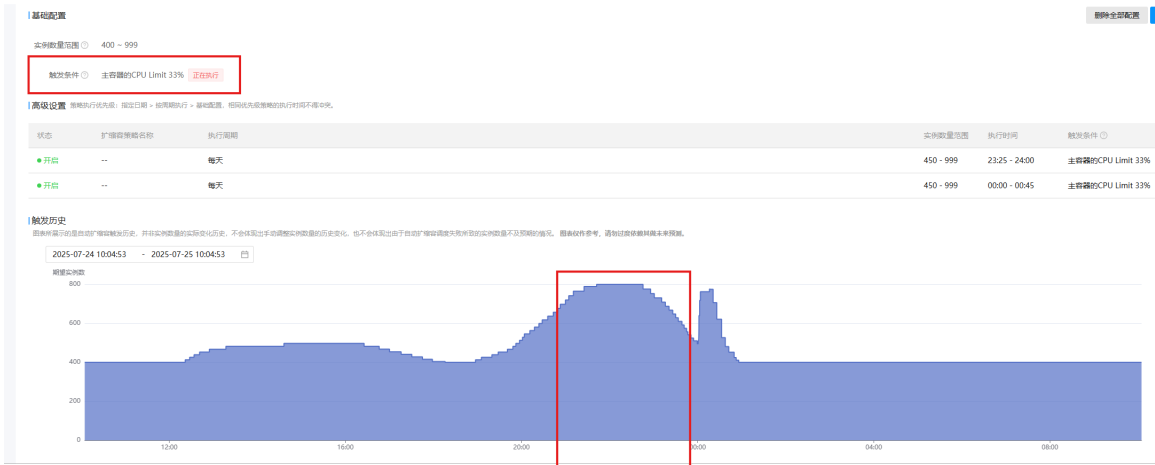


20250725日报

今日工作内容

1. 研究fc_mall_feature的性能优化——使CPU使用率降低；

1、问题背景



问题描述：从上图的自动扩缩容可以发现，当主容器的CPU使用率 $\geq 33\%$ ，就会自动触发扩容，最终导致的现象就是在高峰阶段有近800个实例，即资源总量有 $800 \times 2 = 1600$ 核。现在希望对其性能优化，使其CPU使用率降低，这样就能减少扩容数，使高峰阶段的实例数降低，即CPU核数总量使用减少。

2、研究过程：

a) trpc-go将cpu耗时等监管集成到了yaml配置中，直接使用admin的ip和port即可；

1

admin:

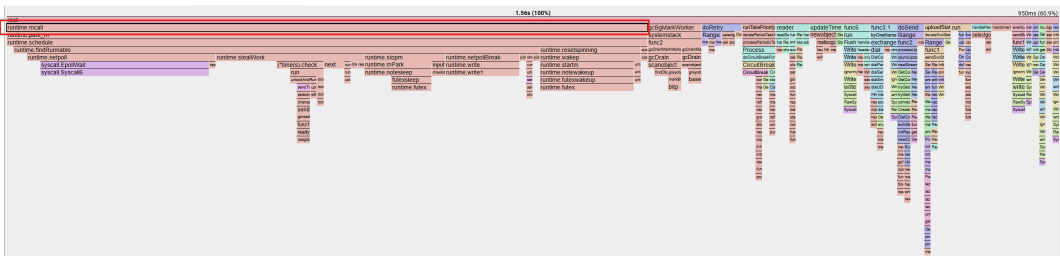
2

nic: eth0

3

port: 8091

b) 使用admin中的ip:port，得到火焰图



从图中可知，cpu主要消耗在了`runtime.schedule`，表示的是调度器在管理协程切换时的开销，也就是大量的消耗都是协程调度和协程切换。切换操作频繁可能是协程数量过多或阻塞操作过多。

c) 在dev环境自己写客户端测试代码，然后对服务进行压测，再利用go tool pprof -seconds 30 http://21.6.68.166:8091/debug/pprof/profile对admin中的端口进行分

析得到火焰图。

- 3、经过和自己思考与岳哥沟通，上述在开发机这种测试方式是不对的，上述直接对本地启动的服务进行cpu耗时分析就不对，因为服务启动后没有上游调用，它的cpu控制权就会交出去，导致此时的大部分cpu耗时都在协程调度上面（即阻塞等待上游调用）。解决的办法就是在现网环境中去测试，然后得到火焰图进行分析。下面在现网环境对fc_mall_feature的一个实例进行的分析，并得到了前50个cpu耗时最多的：

```
(pprof) top 50
Showing nodes accounting for 15.58s, 65.13% of 23.92s total
Dropped 561 nodes (cum <= 0.12s)
Showing top 50 nodes out of 246
```

	flat	flat%	sum%	flat	flat%	sum%	
1.84s	7.69%	7.69%	4.87s	20.36%	runtime.scanobject		
1.69s	7.07%	14.76%	5.83s	24.37%	runtime.mallocgc		
1s	4.18%	18.94%	1.54s	6.44%	runtime.findobject		
0.62s	2.59%	21.53%	0.62s	2.59%	runtime.memclrNoHeapPointers		
0.57s	2.38%	23.91%	0.57s	2.38%	runtime/internal/syscall.Syscall6		
0.53s	2.22%	26.13%	0.53s	2.22%	runtime.(*mspan).base (partial-inline)		
0.50s	2.09%	28.22%	2.70s	11.29%	runtime.mapassign_faststr		
0.45s	1.88%	30.10%	0.45s	1.88%	runtime.nextFreeFast (inline)		
0.44s	1.84%	31.94%	0.58s	2.42%	runtime.(*mspan).writeHeapBitsSmall		
0.41s	1.71%	33.65%	0.41s	1.71%	aeshashbody		
0.34s	1.42%	35.08%	0.36s	1.51%	runtime.(*mspan).heapBitsSmallForAddr		
0.34s	1.42%	36.50%	0.62s	2.59%	runtime.mapaccess2_faststr		
0.32s	1.34%	37.83%	5.24s	21.91%	google.golang.org/protobuf/internal/impl.consumeMapOfMessage		
0.32s	1.34%	39.17%	0.98s	4.08%	runtime.grayobject		
0.32s	1.34%	40.51%	0.36s	1.51%	runtime.spanOf (inline)		
0.31s	1.30%	41.81%	1.02s	4.26%	runtime.evacuate_faststr		
0.31s	1.30%	43.10%	0.39s	1.63%	runtime.typePointers.next		
0.29s	1.21%	44.31%	5.93s	24.79%	google.golang.org/protobuf/internal/impl.(*MessageInfo).unmarshalPointer		
0.29s	1.21%	45.53%	0.29s	1.21%	runtime.memmove		
0.23s	0.96%	46.49%	0.30s	1.25%	strconv.formatBits		
0.22s	0.92%	47.41%	0.22s	0.92%	runtime.typePointers.nextFast (inline)		
0.21s	0.88%	48.29%	6.02s	25.17%	fc_mall_feature/generators/common.(*Engine).calculateItemSide		
0.19s	0.79%	49.08%	0.23s	0.96%	bytes.(*Builder).write		
0.18s	0.75%	49.83%	0.19s	0.79%	runtime.(*gcBits).bitp (inline)		
0.18s	0.75%	50.59%	0.18s	0.75%	runtime.releaseasm (inline)		
0.18s	0.75%	51.34%	0.18s	0.75%	unicode/utf8.Valid		
0.17s	0.71%	52.05%	1.74s	7.27%	encoding/json.arrayEncoder.encode		
0.17s	0.71%	52.76%	0.41s	1.71%	runtime.(*unwinder).resolveInternal		
0.16s	0.67%	53.43%	0.55s	2.30%	fc_mall_feature/generators/fcmallhot/kernels.(*IntGenerator).Generate		
0.16s	0.67%	54.10%	0.16s	0.67%	memqbody		
0.16s	0.67%	54.77%	0.16s	0.67%	runtime.(*mspan).divideByElemSize (inline)		
0.15s	0.63%	55.39%	0.15s	0.63%	internal/abi.(*Type).Kind (inline)		
0.15s	0.63%	56.02%	0.42s	1.76%	reflect.implements		
0.15s	0.63%	56.65%	0.15s	0.63%	runtime.acquirem (inline)		
0.15s	0.63%	57.27%	0.15s	0.63%	runtime.futex		
0.15s	0.63%	57.90%	5.21s	21.78%	runtime.gcdrain		
0.14s	0.59%	58.49%	0.93s	3.89%	runtime.deductAssistCredit		
0.14s	0.59%	59.07%	0.32s	1.34%	runtime.mapaccess2		
0.14s	0.59%	59.66%	0.17s	0.71%	runtime.pageIndexof (inline)		
0.14s	0.59%	60.24%	0.16s	0.67%	runtime.step		
0.13s	0.54%	60.79%	0.35s	1.46%	runtime.pcvvalue		
0.13s	0.54%	61.33%	0.60s	2.51%	runtime.wbBufFlush1		
0.12s	0.5%	61.83%	0.12s	0.5%	encoding/json.appendString[go.shape.string]		
0.12s	0.5%	62.33%	0.15s	0.63%	runtime.spanOfUnchecked (inline)		
0.12s	0.5%	62.83%	0.12s	0.5%	sync.(*entry).load (inline)		
0.11s	0.46%	63.29%	0.13s	0.54%	encoding/json.(*decodeState).rescanLiteral		
0.11s	0.46%	63.75%	0.68s	2.84%	encoding/json.intEncoder		
0.11s	0.46%	64.21%	0.19s	0.79%	reflect.Value.Index		
0.11s	0.46%	64.67%	0.35s	1.46%	runtime.bulkBarrierPreWrite		
0.11s	0.46%	65.13%	0.69s	2.88%	runtime.heapSetType		

分析：图中框出来的是业务代码cpu占比较高的，后续代码还没有具体分析，现在只是得到了这个图；

今日所遇问题：

问题：在开发机直接得到的火焰图进行后续cpu耗时分析是不对的，因为服务需要上游调用；

解决办法：直接在现网环境即可，此时得到的火焰图才能真正刻画问题背景中出现的“超多核”的问题；

明日待办：

1. 重新在现网环境下进行分析fc_mall_feature的各个函数部分的cpu使用情况；